Efficient Scene Flow Learning for Large-scale 3D Point Clouds

Chensheng Peng, Guangming Wang, Xian Wan Lo, Chenfeng Xu,
Masayoshi Tomizuka, Wei Zhan, and Hesheng Wang

***Abstract*— Scene flow estimation from point clouds (PC) is a crucial perception task in autonomous driving and robotics. Nevertheless, point clouds are naturally sparse, unordered and unstructured, resulting in huge challenges when conducting high-resolution scene flow estimation. One of the most notable challenges is the memory-inefficiency and heavy overhead from distance calculation and sorting, involved in commonly used farthest point sampling, KNN and ball query grouping for local feature aggregation. To mitigate this issue, we introduce a projection-based 2D representation of raw 3D point clouds. It is a novel use of projection in the dense 3D scene flow estimation. Unlike previous works which sample small amount of points from the complete point sets, the dense 2D representation 1) preserves most information in the given scene, and 2) brings in significant boost of efficiency. Instead of using 2D CNNs, we utilize 3D point cloud learning method to accomplish large-scale scene flow estimation with one-time inference using prior spatial information from the dense representation. Specifically, by reducing the searching range and complexity of local query, our proposed network achieves single-frame feature extraction and inter-frame correlation for tens of thousands of LiDAR point clouds. Another challenge is that multiple points can be mapped into one grid during projection, causing information loss like voxelization. We present a novel warping projection technique to alleviate such problems when computing the cost volume. The whole framework achieves real time inference. Sufficient experiments demonstrate the efficiency and effectiveness of our method, outperforming the prior-arts on the FlyingThings3D dataset and KITTI Scene Flow dataset.**

I. Introduction

Scene flow represents the point-wise motion of a pair of point clouds, specifically the magnitude and direction of the 3D motion. As a low-level task, 3D scene flow estimation is beneficial to many high-level scene understanding tasks in autonomous driving, such as LiDAR odometry [1], object tracking [2], and semantic segmentation [3].

With the proposal of PointNet [4] and PointNet++ [5], recent studies [6], [7], [8] process raw 3D point clouds and resort to directly learning scene flow from the coordinates of point clouds in 3D space. FlowNet3D [6] builds a flow embedding layer to compute correlation embeddings that can be propagated through up-sampling structure to predict scene flow. PointPWC-Net [8] adopts a patch-to-patch manner to expand the receptive field. HALFLOW [9] proposes a double attentive embedding layer to assign different weights to respective correlated points. Abovementioned attentive techniques create more precise correlation between point clouds, yet lead to a longer running time and increasing burden of computing resources. HPLFlowNet [7] works on minimizing memory and latency through sparse convolution. However, the information loss caused by interpolation onto permutohedral lattice limits its performance.

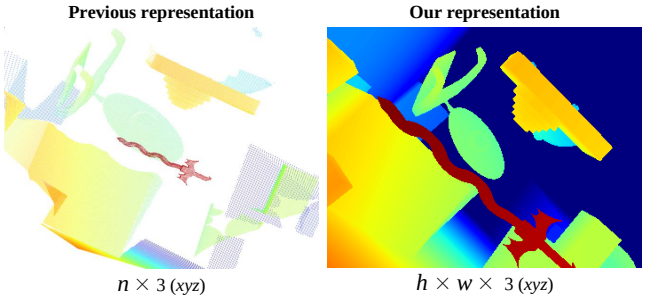

Fig. 1. The new representation form of point clouds by projecting the points onto image plane. Every point in 3D space corresponds to a pixel in image plane, and each pixel contains the coordinate information of the original point.

The most commonly used K Nearest Neighbors (KNN) and ball query algorithms in methods like [6], [7], [8] usually require frequent distance calculation and sorting between point clouds which is memory-inefficient and time-consuming. Therefore, only a limited number of input points are taken because larger scale of input point clouds indicate more GPU memory consumption. To tackle this problem, we introduce a dense representation of point clouds by projecting the raw 3D points ($n \times 3$) onto a 2D plane ($H \times W \times 3$) as shown in Fig. 1. Such representation allows us to take the complete point clouds from input scene. We are able to process them with less memory consumption because more prior spatial information from the 2D form reduces the local query complexity. Based on the 2D representation, we construct an efficient neural network to predict scene flow. Furthermore, we adopt a warping operation in cost volume to refine the predicted coarse flow. However, multiple warped points can be projected into the same grid on 2D plane during refinement. Previous methods [10] merge extra points, but cause information loss which affects the final accuracy. On the contrary, our proposed cost volume module equipped with a novel warping projection technique enables us to avoid

*This work was supported in part by the Natural Science Foundation of China under Grant 62225309, Grant 62073222, Grant U21A20480, and Grant U1913204; in part by the Science and Technology Commission of Shanghai Municipality under Grant 21511101900; and in part by the Open Research Projects of Zhejiang Laboratory under Grant 2022NB0AB01. Corresponding Author: Hesheng Wang.

C, Peng. G. Wang, X. Lo, and H. Wang are with Department of Automation, Key Laboratory of System Control and Information Processing of Ministry of Education, Key Laboratory of Marine Intelligent Equipment and System of Ministry of Education, Shanghai Engineering Research Center of Intelligent Control and Management, Shanghai Jiao Tong University, Shanghai 200240, China.

C, Xu. M. Tomizuka, and W. Zhan are with Department of Mechanical Engineering, University of California, Berkeley, CA 94720, USA.

such information loss caused by merging operations. By achieving more information input from the dense representation and less information loss from the new cost volume, our approach is proved to be able to outperform other methods on the FlyingThings3D dataset [11] and KITTI Scene Flow dataset [12] in terms of effectiveness and efficiency.

The main contributions of this paper are as follows:

- We introduce a dense 2D representation of point clouds to scene flow estimation. By projecting 3D points onto the 2D plane, we take the entire point clouds as input. An efficient network is built to process the input point clouds with more prior spatial information.
- We propose a novel cost volume module which utilizes a warping projection technique, allowing us to compute the cost volume without possible information loss caused by merging points during the refinement process.
- Quantitative results on the synthetic FlyingThings3D dataset and real-world KITTI Scene Flow dataset show that our method outperforms recent SOTA methods both in accuracy and efficiency. Ablation studies verifies the effectiveness of our proposed components.

II. Related work

A. Scene Flow Learning from 3D Data

With the development of deep learning for raw 3D point cloud processing [4], [5], FlowNet3D [6] pioneers in directly processing point clouds and predicts 3D scene flow in an end-to-end fashion. A flow embedding layer is proposed to compute the correlation between a pair of point clouds. PointPWC-Net [8] proposes a patch-to-patch method by considering more than just one point of the first frame during the correlation process, and extends the coarse-to-fine structure from optical flow [13] to scene flow estimation. Inspired by permutohedral lattice [14] and Bilateral Convolutional Layers (BCL) [15], HPLFlowNet [7] proposes DownBCL, UpBCL, and CorrBCL to restore rich information from point clouds. However, the interpolation from points to permutohedral lattice leads to information loss. To assign different weights to the correlated points in a patch, HALFLOW [9] proposes a hierarchical neural network with a double attentive embedding layer. Using optimal transport [16], [17], [18] tools, FLOT [19] achieves competitive performance with much less parameters. The rigidity assumption is widely used in other works [20], [21], [22], [23]. HCRF-Flow [24] formulates the rigidity constraint as a high order term.

These previous works are mostly found to be ignoring the efficiency during scene flow estimation. Therefore, we aim to construct a network that can efficiently process the entire point clouds of a frame with high accuracy.

B. Efficient Learning of Point Clouds

Early works [25], [26] convert the point cloud into voxel grids and process them with 3D Convolutional Neural Networks (CNNs). The information loss happens during voxelization because multiple points can be mapped into one grid, and only one point is kept [27]. Such voxel-based methods are not memory-efficient due to high computational requirements with increasing voxel resolution [28]. Therefore, projection-based methods [29], [30], [31], [32], [33] are proposed to tackle this problem. The 3D point clouds are first projected onto a 2D plane, and then 2D CNNs are applied. DarkNet53 [34] applies a fully convolutional neural network on the projected points after spherical projection. Wang *et al*. [10] proposes a cylindrical projection method in LiDAR odometry. However, projection-based methods usually come with loss of geometry information [27]. Recent point-based methods operate on raw point clouds without a voxelization process. For example, PointNet [4] uses Multi-Layer Perception (MLP) to extract global features directly from raw point clouds. PointNet++ [5] explored the ability to learn local features from neighborhood points. Methods [6], [9], [8] based on PointNet++ are efficient and effective for small-scale point clouds, but the computational cost increases greatly when it comes to large-scale points. YOGO [35] accelerates KNN and ball query algorithms by dividing points into several sub-regions and operating on the tokens extracted from sub-regions.

In this work, we introduce a dense projection form of point clouds. The new form enables us to only calculate the distances between the central point and other points within the given kernel, which significantly reduce the memory and latency during scene flow prediction.

III. Efficient Scene Flow Prediction for Large-scale 3D Point Clouds

Let $PC_1 = \{x_i | x_i \in \mathbb{R}^3\}_{i=1}^{N_1}$ and $PC_2 = \{y_j | y_j \in \mathbb{R}^3\}_{j=1}^{N_2}$ represent the point clouds from a pair of consecutive frames, the goal of scene flow estimation is to predict the motion $F = \{\Delta x_i | \Delta x_i \in \mathbb{R}^3\}_{i=1}^{N_1}$ of every point in PC_1. Local feature aggregation from neighborhood is widely used to extract high-level point features. Such method is inefficient as traditional KNN and ball query algorithms require $\mathcal{O}(n^2)$ complexity where n is the number of points. To achieve efficient learning of dense point clouds, we introduce a dense 2D form of point clouds, which brings more prior spatial information and reduces computational complexity. Moreover, an efficient network with a new cost volume module is constructed to further improve the performance.

A. 2D Representation of Dense Point Clouds

The original 3D point clouds ($n \times 3$) are unordered, where n is the number of points. Such representation provides no prior knowledge and requires much effort to structure the irregular data [27]. To avoid unnecessary calculations, a new representation of point clouds containing more prior spatial information is essential for processing large-scale point clouds efficiently. Inspired by a cylindrical projection technique from [10], we introduce a dense 2D representation form to 3D scene flow estimation task. We conduct pixelization by projecting the raw point clouds (x, y, z) onto the 2D plane (u, v) with a calibration matrix.

Different from other projection-based methods which contain RGB information in the pixel, we fill (u, v) with 3D coordinates (x, y, z) of point cloud, generating an $H \times W \times 3$

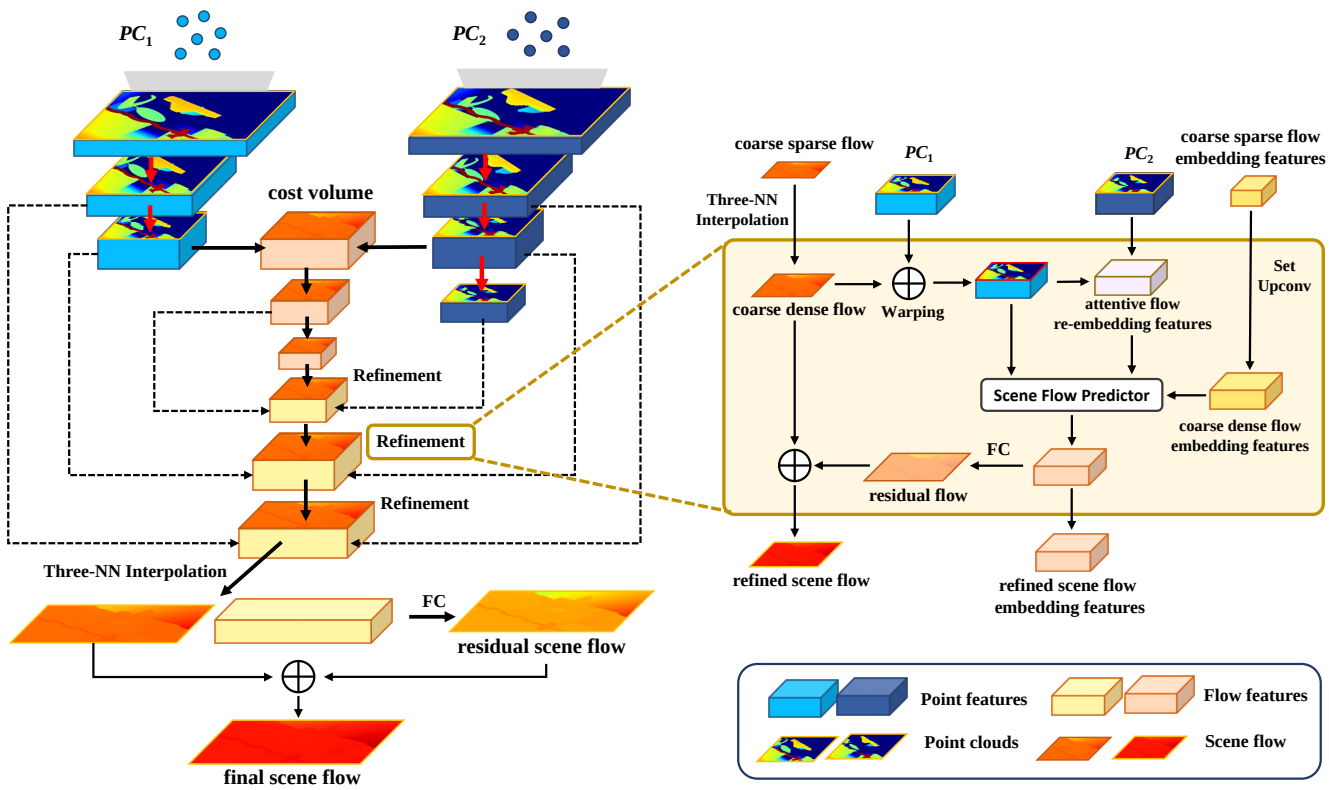

Fig. 2. The structure overview of our proposed network is on the left. The details of the refinement module is on the right. Given a pair of consecutive point clouds, down-sampling, correlation, and up-sampling is performed in order. The network predicts 3D scene flow in a coarse-to-fine manner with the refinement module.

matrix where H and W are height and width of the projected image plane. Not every pixel has its corresponding point in 3D space, therefore we fill empty pixels with $(0, 0, 0)$. Our pixelization technique preserves complete information from raw point clouds compared to the interpolation in [7] and sampling operation in [6], [8], [9]. It allows us to process the entire LiDAR scans of the scene at once. Our scene flow prediction network takes the entire point clouds as input like an image, but with 3D coordinates instead of RGB value in the corresponding pixels.

B. Structure of the Scene Flow Prediction Network

Following the projection technique in Section III-A, each point's coordinate is stored in the corresponding projected pixel position, and all points of the scene are taken as input. Since the resolution of point cloud is currently much higher than randomly sampling a small number of points, an efficient algorithm to reduce the computational cost is in need. Therefore, we construct an efficient 3D scene flow estimation network based on the 2D representation form as shown in Fig. 2. We adopt a hierarchical structure to predict the scene flow in a coarse-to-fine manner. Our proposed network includes the following components: a point cloud encoder, correlation module, scene flow decoder, and a refinement module.

1) Kernel-based Neighbors Grouping: Upon the 2D map of 3D point clouds, a kernel-based grouping technique is used to select neighborhood points. In most cases, two nearby points in 3D space are also close in 2D plane after projection, which helps filter out many inappropriate neighborhood points without extra calculations.

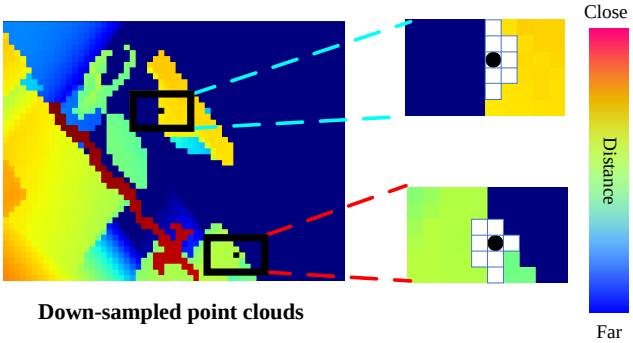

Fig. 3. Kernel-based grouping method. Black pixel denotes the central point, and the black rectangle is the given kernel. The white pixels are selected points using KNN.

As shown in Fig. 3, given the central point p_c with corresponding 2D coordinate (u_c, v_c), a kernel is placed around the center. We can easily locate its neighborhood points within the kernel on 2D plane. The kernel size for grouping is denoted as $k_s = [k_h \times k_w]$, so we only need to search the points inside range: $[u_c - k_w/2 : u_c + k_w/2, v - k_h/2 : v + k_h/2]$ instead of the whole point clouds. This technique is able to reduce the complexity to $\mathcal{O}(n \cdot k_s)$.

It is noteworthy that larger kernel indicates more searching time and reducing the kernel size results in a lower latency. However, if the kernel size is too small, the algorithm cannot find the correct neighboring points. Through experiments, we find that the most appropriate kernel size is usually between $3K \sim 4K$ where K is the number of neighbors to select.

2) Point Cloud Encoder and Scene Flow Decoder: The input point clouds are stored in a form of image. For high-level feature learning, we attempt to aggregate local features from neighboring points to the central point using learnable down-sampling layers.

Given the raw point sets $P = \{x_i | x_i \in \mathbb{R}^3\}_{i=1}^{N}$, we first select a number of central points, generating a point set $P_c = \{x_i^{'} | x_i^{'} \in \mathbb{R}^3\}_{i=1}^{N^{'}}$. The corresponding features of P and P_c are denoted as $\{f_i | f_i \in \mathbb{R}^C\}_{i=1}^{N}$ and $\{f_i^{'} | f_i^{'} \in \mathbb{R}^C\}_{i=1}^{N^{'}}$. The kernel-based grouping method is utilized to select K neighbors for each point $x_i^{'}$.

We then use MLP to extract the neighborhood features $\{(x_{ik}^{'}, f_{ik}^{'}) | x_{ik}^{'} \in \mathbb{R}^3, f_{ik}^{'} \in \mathbb{R}^C\}_{k=1}^{K}$ as follows:

$$e_i^{'} = \underset{k}{\textit{MaxPool}} \ \{\mathbf{MLP}((x_{ik}^{'} - x_i^{'}) \oplus f_{ik}^{'})\}. \tag{1}$$

3) Attentive Feature Correlation of Point Clouds: After obtaining the high-level features of PC_1 and PC_2, we need to compute their motion relationship. We use the attentive cost volume in [9] to learn the hidden motion information between two consecutive frames of point clouds.

Given point clouds $PC_1 = \{(x_i, f_i) | x_i \in \mathbb{R}^3, f_i \in \mathbb{R}^C\}_{i=1}^{N_1}$ and $PC_2 = \{(y_j, g_j) | y_j \in \mathbb{R}^3, g_j \in \mathbb{R}^C\}_{j=1}^{N_2}$, the attentive flow embedding features $E = \{(x_i, e_i) | x_i \in \mathbb{R}^3, e_i \in \mathbb{R}^{C^{'}}\}_{i=1}^{N_1}$ can be obtained from the cost volume module. An attention mechanism is used to weigh different points in the correlated patch. We re-implement the double attentive flow embedding layers under the 2D representation. The embedding features E are used for flow estimation and refinement in the subsequent procedure.

4) Hierarchical Flow Refinement: We adopt the pyramid structure for supervised learning of scene flow estimation. Both the low-level and high-level scene flow is used to compute the training loss. We first predict a flow with the least number of points from the highest-level features. Then level by level, scene flow with higher resolution is predicted through up-sampling and refinement modules. For up-sampling layers, similar operations in equation (1) are performed to obtain $e_i^{'}$ at first. Then, the features $e_i^{'}$ are concatenated with the original features $f_i^{'}$ of central points. Another shared MLP is used to generate up-sampled features.

$$f_i^{''} = \mathbf{MLP}(e_i^{'} \oplus f_i^{'}). \tag{2}$$

After up-sampling operation, we obtain a coarse flow embeddings of more points. Next, a warping operation in Section III-C is adopted to generate the warped features and re-embedding features. A residual scene flow can then be predicted from the warped features, up-sampled features, and re-embedding features through a fully connected layer. Finally, we obtain a finer flow of higher accuracy by adding the estimated residual flow to the coarse scene flow.

C. A New Cost Volume with Warping Projection

Assuming that $p \in \mathbb{R}^3$ of PC_1 moves into $q \in \mathbb{R}^3$ of PC_2, their corresponding 2D coordinates are (u_1, v_1) and (u_2, v_2). When the magnitude of flow f_{gt} between p and q is great, p exhibits a large motion across frames, and (u_1, v_1) could be far away from (u_2, v_2). This requires a large searching kernel if we aim to find (u_2, v_2) correctly in the bounding kernel around (u_1, v_1). However, enlarging the kernel size could cause an increase of searching time and violates our intention to shorten the computational cost. To solve this problem, we integrate the warping idea to our network.

Following the coarse-to-fine manner, a coarse flow f_c is predicted as described in Section III-B.4. Previous works [40], [8] directly add it to PC_1, generating $PC_{1,warped} = \{x_{i,warped} | x_{i,warped} \in \mathbb{R}^3\}_{i=1}^{N}$, which is closer to PC_2 than the original PC_1. Then cost volume is computed between $PC_{1,warped}$ and PC_2 to predict a residual flow f_Δ. With level-by-level refinement, the warped points become closer and closer to PC_2. However, abovementioned warping idea cannot be applied to our method directly under the 2D representation. This is because after adding f_c, multiple close points among $PC_{1,warped}$ can be mapped into one grid. A demand for merging points will cause information loss and performance degeneration.

In our proposed network, we present a new cost volume module equipped with a warping projection technique to tackle this problem. The cost volume is not directly computed between PC_1 and PC_2 based on their pixel position (u_1, v_1) and (u_2, v_2). Instead, after obtaining $PC_{1,warped}$, we first project $PC_{1,warped}$ onto the image plane and obtain the corresponding 2D position (u_{warped}, v_{warped}). Rather than merging multiple points with same 2D positions, we create a $h \times w \times 2$ matrix to store (u_{warped}, v_{warped}) of every warped point in $PC_{1,warped}$. (u_{warped}, v_{warped}) acts as an indexing variable between (u_1, v_1) and (u_2, v_2) when performing correlation. The coordinates of $PC_{1,warped}$ are stored in (u_1, v_1), with the size $h \times w \times 3$. All local calculations run in parallel. For each warped point stored at (u_1, v_1), a kernel is placed on PC_2 $(h \times w \times 3)$ around the corresponding index position (u_{warped}, v_{warped}). Then, the distances between $PC_{1,warped}$ and PC_2 within the kernel are calculated. Distant PC_2 points are filtered out and K nearest neighbors are kept. Local feature aggregation from the K points of PC_2 to the central warped point at (u_1, v_1) update the features of $PC_{1,warped}$.

When computing cost volume, every warped point has a corresponding 2D coordinate. Our proposed warping projection method can avoid losing points caused by merging operation and does not require a large kernel. This means that complete point clouds can be reserved and processed.

IV. Experiments

A. Datasets and Evaluation Metrics

We conduct experiments on the synthetic FlyingThings3D dataset [11] and real-world KITTI Scene Flow dataset [12]. Following the preprocessing methods described in [8], [7],

TABLE I

QUANTITATIVE RESULTS COMPARED WITH RECENT METHODS ON FLYINGTHINGS3D DATASET [11] AND KITTI SCENE FLOW DATASET [12]. QUARTER MEANS TRAINING THE MODEL ON ONE FOURTH OF THE DATA. COMPLETE MEANS USING THE COMPLETE DATASET FOR TRAINING. ↑ DENOTES BETTER PERFORMANCE WITH HIGHER VALUE, AND ↓ IS THE OPPOSITE.

Dataset	Method	Training Data	EPE3D ↓	Acc3DS ↑	Acc3DR ↑	Outliers3D ↓	EPE2D ↓	Acc2D ↑
FT3D	ICP [36]	No	0.4062	0.1614	0.3038	0.8796	23.2280	0.2913
	FlowNet3D [6]	Quarter	0.1136	0.4125	0.7706	0.6016	5.9740	0.5692
	SPLATFlowNet [37]	Quarter	0.1205	0.4197	0.7180	0.6187	6.9759	0.5512
	HPLFlowNet [7]	Quarter	0.0804	0.6144	0.8555	0.4287	4.6723	0.6764
	HPLFlowNet [7]	Complete	0.0696	-	-	-	-	-
	PointPWC-Net [8]	Complete	0.0588	0.7379	0.9276	0.3424	3.2390	0.7994
	HALFlow [9]	Quarter	0.0511	0.7808	0.9437	0.3093	2.8739	0.8056
	HALFlow [9]	Complete	0.0492	0.7850	0.9468	0.3083	2.7555	0.8111
	FLOT [19]	Complete	0.0520	0.7320	0.9270	0.3570	-	-
	HCRF-Flow [24]	Quarter	0.0488	0.8337	0.9507	0.2614	2.5652	0.8704
	PV-RAFT [38]	Complete	0.0461	0.8169	0.9574	0.2924	-	-
	FlowStep3D [39]	Complete	0.0455	0.8162	0.9614	0.2165	-	-
	Ours	Complete	**0.0388**	**0.8739**	**0.9637**	**0.2062**	**2.2042**	**0.8762**
KITTI	ICP [36]	No	0.5181	0.0669	0.1667	0.8712	27.6752	0.1056
	FlowNet3D [6]	Quarter	0.1767	0.3738	0.6677	0.5271	7.2141	0.5093
	SPLATFlowNet [37]	Quarter	0.1988	0.2174	0.5391	0.6575	8.2306	0.4189
	HPLFlowNet [7]	Quarter	0.1169	0.4783	0.7776	0.4103	4.8055	0.5938
	HPLFlowNet [7]	Complete	0.1113	-	-	-	-	-
	PointPWC-Net [8]	Complete	0.0694	0.7281	0.8884	0.2648	3.0062	0.7673
	HALFlow [9]	Quarter	0.0692	0.7532	0.8943	0.2529	2.8660	0.7811
	HALFlow [9]	Complete	0.0622	0.7649	0.9026	0.2492	2.5140	0.8128
	FLOT [19]	Complete	0.0560	0.7550	0.9080	0.2420	-	-
	HCRF-Flow [24]	Quarter	0.0531	**0.8631**	0.9444	0.1797	2.0700	0.8656
	PV-RAFT [38]	Complete	0.0560	0.8226	0.9372	0.2163	-	-
	FlowStep3D [39]	Complete	0.0546	0.8051	0.9254	**0.1492**	-	-
	Ours	Complete	**0.0437**	0.8198	**0.9676**	0.2025	**1.4123**	**0.9176**

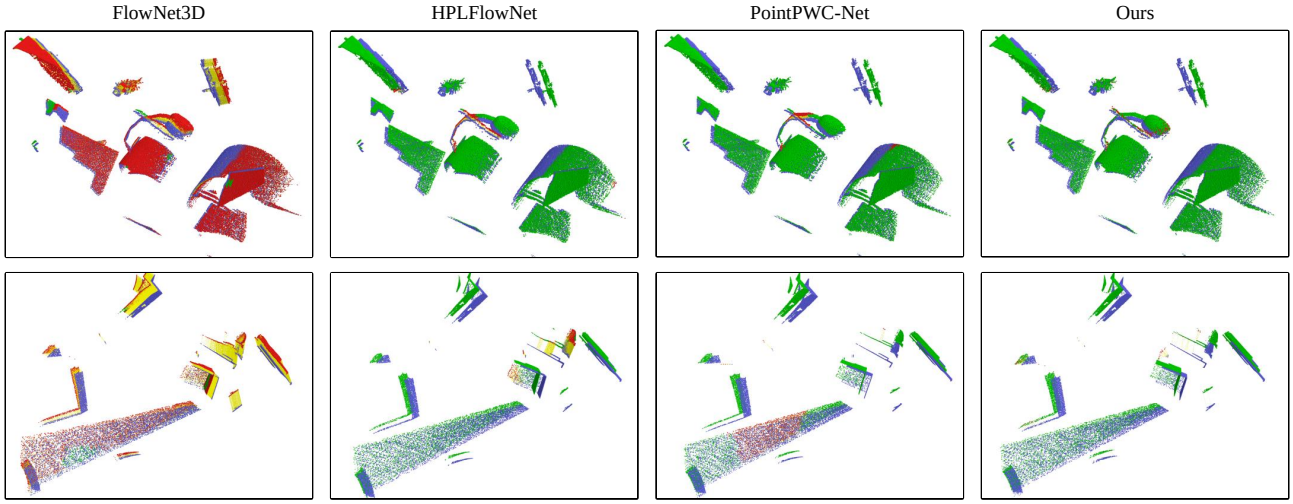

Fig. 4. Qualitative visualization results of scene flow estimation. We compare our methods with FlowNet3D [6], HPLFlowNet [7] and PointPWC-Net [8]. Blue points represent PC_1. Points with accurate predictions are in green, and wrong points are red. The yellow lines connect PC_1 with wrongly predicted points. The accuracy is measured using ACC3DR.

[9], the point clouds and ground truth scene flow are constructed from the depth map and optical flow. FlyingThings3D dataset consists of 19640 training samples and 3824 testing samples. We remove points with depth exceeding 35m like [8], [7], [9]. Using the 2D representation described in Section III-A, the 3D points are first projected to 2D plane using the given calibration matrix from the FT3D dataset. The input size of point cloud is $270 \times 480 \times 3$. KITTI Scene Flow dataset contains 200 training samples and 200 testing samples. Since no ground truth flow for testing samples is provided, we separate the training dataset into training and validation split. Points with depth exceeding 35m and points on height less than 0.3m are removed. The size of input point cloud after projection is $375 \times 1242 \times 3$. The model trained on FlyingThings3D is fine-tuned on KITTI dataset.

For a fair comparison, we adopt the following evaluation metrics as [6], [8], [7], [9]: EPE3D, Acc3DS, Acc3DR, Outliers3D for scene flow and EPE2D, Acc2D for optical

TABLE II

COMPARISON OF MODEL SIZE AND RUNNING TIME.

Methods	Input point number	Size (MB)	Time (ms)
HPLFlowNet	8192	231.8	93.67
HPLFlowNet	49152	231.8	132.58
PointPWC-Net	8192	30.1	100.05
PointPWC-Net	49152	30.1	2643.61
Ours	56269 (270×480)	**19.1**	**52.91**

flow evaluation. In this work, the optical flow for evaluation is obtained by projecting the 3D scene flow onto image plane.

B. Implementation Details

All experiments are conducted on NVIDIA Quadro RTX 8000 GPU with PyTorch 1.10.0. We use Adam [41] as our optimizer with $\beta_1 = 0.9, \beta_2 = 0.999$. The initial learning rate of 0.001 decays exponentially with a decaying rate $\gamma = 0.8$, and the decaying step is 80.

Our designed network adopts the hierarchical structure, where both low-level and high-level scene flow are predicted. Therefore, we adopt a multi-scale supervised manner to train the model. Let $F^l = \{f_i^l | f_i^l \in \mathbb{R}^3\}_{i=1}^{N_l}$ be the predicted scene flow at level l, and $\hat{F}^l = \{\hat{f}_i^l | \hat{f}_i^l \in \mathbb{R}^3\}_{i=1}^{N_l}$ be the corresponding ground truth scene flow. The training loss is calculated as follows:

$$Loss = \sum_{l=1}^{4} w_l \frac{1}{N_l} \sum_{i=1}^{N_l} \left\| \hat{f}_i^l - f_i^l \right\|_2, \tag{3}$$

where N_l is the number of valid points at level l. w_l denotes the weighting factor of losses at different levels, and $w_1 = 0.02, w_2 = 0.04, w_3 = 0.08, w_4 = 0.16$.

C. Experimental Results

1) Comparison with State-of-the-Arts: The quantitative results on FlyingThings3D dataset and KITTI Scene Flow dataset are listed in Table I. We compare our method with other baselines in terms of the evaluation metrics described in Section IV-A. Results show that our method outperforms other methods both in 3D metrics and 2D metrics.

We believe that the improvement comes from two aspects, 1) The dense 2D representation. Previous methods usually take 8192 points as input, which is much smaller than the total number of points in a scene. The sampling operation leads to a sparse representation, which is unfavorable because of losing object details, while the dense representation provides a more precise structure of the scene. Through the 2D representation form, we reserve the complete information of the scene without losing points. 2) The new cost volume with warping projection technique. In previous warping method, the warped points are directly projected onto the 2D plane. Severe loss of information could happen, as multiple points projected into one grid are merged. Our new warping projection method enables us to avoid such case by storing the corresponding projected 2D coordinates. In short, more input information and less information loss contribute to the improvements of our approach.

TABLE III

ABLATION STUDY OF REPRESENTATION FORMS

Representation	Input point number	EPE3D	EPE2D
Sparse ($n \times 3$)	8192	0.0475	2.5885
Dense ($H \times W \times 3$)	56269 (270×480)	**0.0388**	**2.2042**

TABLE IV

ABLATION STUDY OF WARPING PROJECTION

Warping Projection	EPE3D	Acc3DS	Acc3DR	Outlier
	0.0418	0.8601	0.9624	0.2434
✓	**0.0388**	**0.8739**	**0.9637**	**0.2062**

The qualitative results shown in Fig. 4 demonstrate that our method achieve prediction with least errors. We also compare the running time and size of our models with other methods as illustrated in Table II, showing that our method outperforms others in terms of efficiency. Since the number of valid points varies across frames, we count the average number of valid points in the validation dataset.

D. Ablation Study

We conduct a series of ablation studies to verify the effectiveness of each component of the network.

1) Dense or Sparse Representation: We compare the performances of our method with or without the new 2D dense representation of point clouds. For experiments without the 2D dense representation, following [8], [9], we randomly select 8192 points and take them as input to the neural network. As adopting 2D representation allows us to take more information as input, the model using 2D dense representation outperforms the sparse 3D representation as shown in Table III.

2) Warping Projection in Cost Volume: For experiments without our proposed warping projection technique, we directly project the PC_{warped} onto the 2D plane and merge multiple points projected into the same grid. The flow embedding features for the merged points are obtained from interpolation. Results in Table IV demonstrate that the new warping projection technique are useful in avoiding information loss and thus improve the model's performance.

V. CONCLUSION

In this paper, we introduce a dense representation form of point clouds, which enables us to take the entire point clouds of the scene as input and estimate the scene flow at once. Moreover, a novel cost volume module based on warping projection is proposed, allowing us to reserve the complete information of the raw point clouds without losing details. With more information input and less information loss, our method can outperform recent methods. We hope our work will serve as a baseline for future work on efficient point cloud processing and scene flow estimation.

REFERENCES

[1] G. Wang, X. Wu, Z. Liu, and H. Wang, "Pwclo-net: Deep lidar odometry in 3d point clouds using hierarchical embedding mask optimization," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 15 910–15 919.

[2] S. Wang, Y. Sun, C. Liu, and M. Liu, "Pointtracknet: An end-to-end network for 3-d object detection and tracking from point clouds," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3206–3212, 2020.

[3] X. Liu, M. Yan, and J. Bohg, "Meteornet: Deep learning on dynamic 3d point cloud sequences," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 9246–9255.

[4] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "Pointnet: Deep learning on point sets for 3d classification and segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 652–660.

[5] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "Pointnet++: Deep hierarchical feature learning on point sets in a metric space," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5099–5108.

[6] X. Liu, C. R. Qi, and L. J. Guibas, "Flownet3d: Learning scene flow in 3d point clouds," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 529–537.

[7] X. Gu, Y. Wang, C. Wu, Y. J. Lee, and P. Wang, "Hplflownet: Hierarchical permutohedral lattice flownet for scene flow estimation on large-scale point clouds," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 3254–3263.

[8] W. Wu, Z. Wang, Z. Li, W. Liu, and L. Fuxin, "Pointpwc-net: A coarse-to-fine network for supervised and self-supervised scene flow estimation on 3d point clouds," *arXiv preprint arXiv:1911.12408*, 2019.

[9] G. Wang, X. Wu, Z. Liu, and H. Wang, "Hierarchical attention learning of scene flow in 3d point clouds," *IEEE Transactions on Image Processing*, vol. 30, pp. 5168–5181, 2021.

[10] G. Wang, X. Wu, S. Jiang, Z. Liu, and H. Wang, "Efficient 3d deep lidar odometry," *arXiv preprint arXiv:2111.02135*, 2021.

[11] N. Mayer, E. Ilg, P. Hausser, P. Fischer, D. Cremers, A. Dosovitskiy, and T. Brox, "A large dataset to train convolutional networks for disparity, optical flow, and scene flow estimation," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 4040–4048.

[12] M. Menze, C. Heipke, and A. Geiger, "Object scene flow," *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 140, pp. 60–76, 2018.

[13] D. Sun, X. Yang, M.-Y. Liu, and J. Kautz, "Pwc-net: Cnns for optical flow using pyramid, warping, and cost volume," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 8934–8943.

[14] A. Adams, J. Baek, and M. A. Davis, "Fast high-dimensional filtering using the permutohedral lattice," in *Computer graphics forum*, vol. 29, no. 2. Wiley Online Library, 2010, pp. 753–762.

[15] V. Jampani, M. Kiefel, and P. V. Gehler, "Learning sparse high dimensional filters: Image filtering, dense crfs and bilateral neural networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 4452–4461.

[16] H. Petric Maretic, M. El Gheche, G. Chierchia, and P. Frossard, "Got: an optimal transport framework for graph comparison," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[17] G. Peyré, M. Cuturi, and J. Solomon, "Gromov-wasserstein averaging of kernel and distance matrices," in *International Conference on Machine Learning*. PMLR, 2016, pp. 2664–2672.

[18] V. Titouan, N. Courty, R. Tavenard, and R. Flamary, "Optimal transport for structured data with application on graphs," in *International Conference on Machine Learning*. PMLR, 2019, pp. 6275–6284.

[19] G. Puy, A. Boulch, and R. Marlet, "Flot: Scene flow on point clouds guided by optimal transport," in *European conference on computer vision*. Springer, 2020, pp. 527–544.

[20] C. Vogel, K. Schindler, and S. Roth, "Piecewise rigid scene flow," in *Proceedings of the IEEE International Conference on Computer Vision*, 2013, pp. 1377–1384.

[21] ——, "3d scene flow estimation with a piecewise rigid scene model," *International Journal of Computer Vision*, vol. 115, no. 1, pp. 1–28, 2015.

[22] M. Menze and A. Geiger, "Object scene flow for autonomous vehicles," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 3061–3070.

[23] W.-C. Ma, S. Wang, R. Hu, Y. Xiong, and R. Urtasun, "Deep rigid instance scene flow," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 3614–3622.

[24] R. Li, G. Lin, T. He, F. Liu, and C. Shen, "Hcrf-flow: Scene flow from point clouds with continuous high-order crfs and position-aware flow embedding," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 364–373.

[25] C. B. Choy, D. Xu, J. Gwak, K. Chen, and S. Savarese, "3d-r2n2: A unified approach for single and multi-view 3d object reconstruction," in *European conference on computer vision*. Springer, 2016, pp. 628–644.

[26] G. Riegler, A. Osman Ulusoy, and A. Geiger, "Octnet: Learning deep 3d representations at high resolutions," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 3577–3586.

[27] H. Tang, Z. Liu, S. Zhao, Y. Lin, J. Lin, H. Wang, and S. Han, "Searching efficient 3d architectures with sparse point-voxel convolution," in *European conference on computer vision*. Springer, 2020, pp. 685–702.

[28] Z. Liu, H. Tang, Y. Lin, and S. Han, "Point-voxel cnn for efficient 3d deep learning," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[29] A. Boulch, B. L. Saux, and N. Audebert, "Unstructured point cloud semantic labeling using deep segmentation networks," in *Proceedings of the Workshop on 3D Object Retrieval*. Eurographics Association, 2017, p. 17–24.

[30] Z. Wang, W. Zhan, and M. Tomizuka, "Fusing bird's eye view lidar point cloud and front view camera image for 3d object detection," in *2018 IEEE intelligent vehicles symposium (IV)*. IEEE, 2018, pp. 1–6.

[31] B. Wu, A. Wan, X. Yue, and K. Keutzer, "Squeezeseg: Convolutional neural nets with recurrent crf for real-time road-object segmentation from 3d lidar point cloud," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 1887–1893.

[32] B. Wu, X. Zhou, S. Zhao, X. Yue, and K. Keutzer, "Squeezesegv2: Improved model structure and unsupervised domain adaptation for road-object segmentation from a lidar point cloud," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 4376–4382.

[33] C. Xu, B. Wu, Z. Wang, W. Zhan, P. Vajda, K. Keutzer, and M. Tomizuka, "Squeezesegv3: Spatially-adaptive convolution for efficient point-cloud segmentation," in *European Conference on Computer Vision*. Springer, 2020, pp. 1–19.

[34] J. Behley, M. Garbade, A. Milioto, J. Quenzel, S. Behnke, C. Stachniss, and J. Gall, "Semantickitti: A dataset for semantic scene understanding of lidar sequences," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 9297–9307.

[35] C. Xu, B. Zhai, B. Wu, T. Li, W. Zhan, P. Vajda, K. Keutzer, and M. Tomizuka, "You only group once: Efficient point-cloud processing with token representation and relation inference module," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 4589–4596.

[36] P. J. Besl and N. D. McKay, "Method for registration of 3-d shapes," in *Sensor fusion IV: control paradigms and data structures*, vol. 1611. Spie, 1992, pp. 586–606.

[37] H. Su, V. Jampani, D. Sun, S. Maji, E. Kalogerakis, M.-H. Yang, and J. Kautz, "Splatnet: Sparse lattice networks for point cloud processing," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 2530–2539.

[38] Y. Wei, Z. Wang, Y. Rao, J. Lu, and J. Zhou, "Pv-raft: point-voxel correlation fields for scene flow estimation of point clouds," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 6954–6963.

[39] Y. Kittenplon, Y. C. Eldar, and D. Raviv, "Flowstep3d: Model unrolling for self-supervised scene flow estimation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 4114–4123.

[40] G. Wang, Y. Hu, X. Wu, and H. Wang, "Residual 3-d scene flow learning with context-aware feature extraction," *IEEE Transactions on Instrumentation and Measurement*, vol. 71, pp. 1–9, 2022.

[41] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *3rd International Conference on Learning Representations, ICLR*, 2015.